- **Multi-token Prediction.** This baseline changes the objective from next-token prediction, to multiple token prediction. Given the graph description, start and goal, $[\mathcal{E} \mid n_1, n_l]$ the model needs to predict the path $[n_1, n_2, n_3, \ldots n_l]$ in a single forward pass.

D TINYSTORIES EXPERIMENTS

We train all models on a single A100 / H100 GPU with 80GB memory.

D.1 TRAINING

The Belief State Transformer's encoders have the following settings: $n_{\text{layers}} = 8$, blocks with embedding dimension $e_{\text{dim}} = 768$, and $n_{\text{heads}} = 8$. The textheads $\mathsf{T}_n, \mathsf{T}_p$ are implemented as a single 3-layer MLP with dimensionality 512 and ReLU activations that outputs two predictions. The total model size is 80 million parameters.

The FIM baseline trains a forward only transformer with the following settings: $n_{\text{layers}} = 12$, blocks with embedding dimension $e_{\text{dim}} = 768$, and $n_{\text{heads}} = 8$. The total model size is 85 million parameters.

The FIM baseline's training is simple—for a given prefix $x_{1:t}$, we prepend a random suffix $x_{k:T}$ for $k \sim \mathcal{U}(t+2, T)$ and train the transformer to predict x_{t+1}. We create as many prefix-suffix inputs as the GPU memory allows. Even then, for longer sequences, it becomes increasingly intractable for the FIM baseline to train on all possible prefix-suffix inputs.

The Belief State Transformer on the other hand, is able to train on all $O(T^2)$ possible prefix-suffix combinations for a sequence of length T, by carefully accumulating text head gradients over all pairs before computing the forward and backward encoder gradients. Without such a scheme, training goes from a few hours to multiple days. As a result, the Belief State Transformer is only around $3-5\times$ slower than the FIM baseline's suffix-sampling update, yet is able to train on all $O(T^2)$ possible prefix-suffix inputs.

We trained the Belief State transformer on 1 epoch of the TinyStories dataset with 2.7M stories, with a batch size of 256. The training takes around 5 hours for 1 epoch. Because the FIM baseline goes through an epoch more quickly than the Belief State transformer, we allow it to run for multiple epochs with a max training time of 5 hours, and use early stopping on the validation loss to select the best checkpoint.

Algorithm 2 Beam Search

Require: text $x_{1:t}$, goal $x_{t+k:T}$, number of steps K, beams n
Priority Queue $Q_0 \leftarrow (1, \emptyset)$
for $k = 1$ to $K-1$ **do**
  for $j = 1$ to n **do**
    if $|Q_{k-1}| > 0$ **then**
      (priority r, sequence u) $\leftarrow$ pop(Q_{k-1})
      for Possible $\tilde{x}$ **do**
        $Q_k \leftarrow (r\mathsf{T}_n(\tilde{x} \mid \mathsf{F}(x_{t+k:T} + x_{1:t} + u)), u + \tilde{x})$
Output: top(n, Q_K)

The FIM model uses beam search (Graves, 2012) to find high probability sequences at inference time. At a high level, beam search only examines and expands upon a fixed amount of paths during search to keep search tractable. To keep the inference time resources comparable, We use the same number of paths as trajectories we generate in Algorithm 1, which is 120.

D.2 EVALUATION

We evaluate the models on 100 held out stories from the TinyStories dataset. For each story, we use the first 50 tokens for the prompt, and last 100 tokens for the suffix. The choice of 100 for the suffix length is due to the varying length of the stories - shorter stories would often just receive padding tokens as the suffix if the suffix window wasn't long enough.

We use GPT4 to judge the quality of generated text. Given two generated stories, one from each model, we ask GPT4 to analyze the stories and output a recommendation on which story is superior. The stories are anonymized and shuffled to prevent information leakage about its author before we feed it to GPT4. For every evaluation, we shuffle the story ordering once, and repeat the the evaluation 3 times for a total of 6 GPT4 evaluations for every story. Since we evaluate across 100 held out stories, this amounts to 600 total queries to GPT4. We use the OpenAI API to automate the process.

Here is an example input we feed into GPT4.

> **System Prompt:** *You are evaluating two story generation models. First, analyze the outputs and note their properties. Then, make an argument for why one is better than another, or say that both are roughly equal.*
>
> Story 1:
>
> Once upon a time, in a warm and sunny place, there was a big pit. A little boy named Tom liked to play near the pit. One day, Tom lost his red ball. He was very sad. Tom asked his friend, Sam, "Did you see my red ball?" Sam said, "No, but let's look together." They looked and looked, but they could not find the red ball. Tom was very sad. Then, a big bird came and dropped the red ball. The bird had the red ball in its beak. Tom and Sam were so happy! They said, "Thank you, bird!" The bird flew away, and Tom played with his red ball all day.
>
> Story 2:
>
> Once upon a time, in a warm and sunny place, there was a big pit. A little boy named Tom liked to play near the pit. One day, Tom lost his red ball. He was very sad. Tom asked his friend, Sam, for help. "Sam, can you help me find my red ball?" Sam said, "Yes, I will help you." They looked and looked, but they could not find the red ball. Just when they were about to give up, a big bird flew down from the sky. The bird had the red ball in its beak! The bird dropped the ball into the pit. Tom and Sam were so happy. They thanked the bird and played with the red ball

Then, GPT4 responds with:

> **Analysis of Outputs:**
>
> Story 1:
>
> **Grammar and Style:** - Simple and clear sentences, with correct grammar throughout. - Repetitive usage of "Tom was very sad". - "Tom and Sam were so happy!" creates a clear and optimistic conclusion.
>
> **Narrative Flow:** - Introduction: Sets the scene with Tom and the pit. - Conflict: Tom loses his red ball. - Resolution: A big bird returns the ball. - Conclusion: Ends with Tom playing all day, providing a satisfying closure.
>
> Story 2:
>
> **Grammar and Style:** - Generally grammatically correct. - Uses a more varied sentence structure. - Dialogue includes direct speech ("Sam, can you help me find my red ball?"), adding a personal touch.
>
> **Narrative Flow:** - Introduction: Similar setup with Tom and the pit. - Conflict: Tom loses his red ball, seeks Sam's help. - Resolution: Big bird returns the ball, but drops it into the pit. - Conclusion: The story seems to end abruptly without further explanation about retrieving the ball from the pit.
>
> Argument for Which Story is Better:
>
> **Conclusion Analysis:** - **Story 1** concludes clearly with Tom playing all day, which establishes a definite and satisfying ending for the reader. - **Story 2** leaves a gap in logic—though the bird returns the ball, it drops it into the pit, and we don't see how Tom and Sam retrieve it. The absence of this information feels like an incomplete ending.
>
> **Grammar and Narrative:** - Both stories are grammatically correct and maintain simple language, making them suitable for young readers. - Story 1 employs more repetition, which could be beneficial for reinforcement but potentially tedious for some readers. Story 2 introduces more varied dialogue which might be more engaging.
>
> **Overall Conclusion:** - Given that Story 1 has a logical setup, development, and conclusion, **Story 1** is the better option. It provides a clear, satisfying end where Tom is happy and continues to play. Story 2, while containing more engaging dialogue, fails to wrap up the narrative logically, leaving the reader with lingering questions about the outcome.

As seen in the example above, GPT4 is able to consider multiple factors about the story, from low level grammar and vocabulary choices, to higher level concepts like cohesiveness and logical flow, and uses these factors to make its final recommendation.

D.3 Model Outputs

Here are additional stories generated by the Belief State transformer in the goal-conditioned setting.

> **Story 1:** You don't have to be scared of the loud dog, I'll protect you". The mole felt so safe with the little girl. She was very kind and the mole soon came to trust her. He was so gentle and kind. The little girl was so happy to have a new friend. She gave the mole a big hug and they played together all day. They ran around the garden and laughed and had lots of fun. When it was time to go home, the little girl hugged the mole and said goodbye. The mole was so happy to have a friend like her and they promised to always be friends.
>
> **Story 2:** Once upon a time, in a warm and sunny place, there was a big pit. A little boy named Tom liked to play near the pit. One day, Tom lost his red ball. He was very sad. Tom asked his friend, Sam, "Did you see my red ball?" Sam said, "No, but let's look together." They looked near the pit and found the red ball. Tom was very happy. He said, "Thank you, Sam!" But then, a big dog came and took the red ball. Tom and Sam were scared. They wanted the ball back. They ran after the dog. The dog ran into the pit and disappeared. Tom and Sam were sad. They went into the pit to search. It was dark and scary. They could not find the ball. They tried to get out, but the pit was too deep. Tom and

Sam were stuck in the pit. They called for help, but no one could hear them. They were sad and scared, and they never got out of the pit.

Story 3: Once upon a time there was a little girl named Lucy. She loved to go to the store to buy sweets with her mom and dad. On this special day, Lucy entered the store with her mom. Lucy saw a big, red ball. She asked her mom, "Can I have the ball, please?" Her mom said, "No, Lucy. We don't have enough money today. We have to go home and get a new one." Lucy was sad, but she understood. She asked her mom, "Can we get a new ball?" Her mom smiled and said, "Yes, we can get a new one. But first, we have to be peaceful and kind to her. Have patience and understanding. Together, you can both be happy!" So, Lucy smiled at the girl and said, "Can we play together?" The little girl softened and smiled back. She agreed to share the toy and even let Lucy have a turn first. Lucy and the little girl played together happily.

And some outputs for the FIM baseline, over the same prefix / suffix pairs from above.

Story 1: You don't have to be scared of the loud dog, I'll protect you". The mole felt so safe with the little girl. She was very kind and the mole soon came to trust her. He said, "Don't be scared, I'm here to protect you". The little girl was so happy and thanked the mole for keeping her safe. From then on, the little girl and the mole were the best of friends.

Story 2: Once upon a time, in a warm and sunny place, there was a big pit. A little boy named Tom liked to play near the pit. One day, Tom lost his red ball. He was very sad. Tom asked his friend, Sam, for help. "Sam, can you help me find my red ball?" Tom said. Sam said, "Yes, I will help you." They looked and looked, but they could not find the red ball. They were very sad. Then, a big bird flew down from the sky. The bird had the red ball in its beak! The bird dropped the ball into the pit. Tom and Sam were so happy. They said, "Thank you, bird!"They went into the pit to search. It was dark and scary. They could not find the ball. They tried to get out, but the pit was too deep. Tom and Sam were stuck in the pit. They called for help, but no one could hear them. They were sad and scared, and they never got out of the pit.

Story 3: Once upon a time there was a little girl named Lucy. She loved to go to the store to buy sweets with her mom and dad. On this special day, Lucy entered the store with her mom and dad. When they arrived at the store, Lucy was so excited to see all the different kinds of sweets. She couldn't believe her eyes when she saw all the different kinds of sweets. She couldn't believe her eyes when she saw all the different kinds of sweets. Lucy's mom smiled at her and said, " to be peaceful and kind to her. Have patience and understanding. Together, you can both be happy!" So, Lucy smiled at the girl and said, "Can we play together?" The little girl softened and smiled back. She agreed to share the toy and even let Lucy have a turn first. Lucy and the little girl played together happily.

E TRAINING THE BELIEF STATE TRANSFORMER

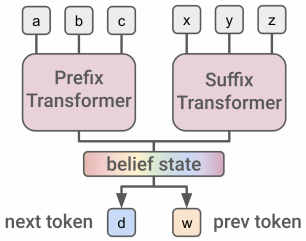


Figure 10: BST input and outputs.

The belief state transformer, at a high level, takes in a prefix $x_{1:i}$, a suffix $x_{i+k:T}$, and predicts the token right after the prefix, and the token right before the suffix. See Figure 10 for an example.

To create training data from a sequence $x_{1:T}$, all valid prefix-suffix pairs are created from the sequence. We define a valid prefix-suffix pair as follows.

Let the prefix be $x_{1:i}$. Then a suffix $x_{i+k:T}$ is valid for all $k \geq 2$. This condition ensures that there is at least one token that separates the prefix and suffix, which makes it feasible to predict tokens in between the prefix and suffix.

For a sequence n tokens long, there are $O(T^2)$ possible valid prefix-suffix pairs. See Figure 11 for example training inputs and their targets.

We present pytorch pseudocode illustrating a simplified implementation of the belief state transformer (Figure 12). Additionally, we present a slightly more complex version which computes and accumulates the gradients for the text head MLP before backpropagating gradients into the encoders (Figure 13). This implementation is more computationally efficient as it avoids backpropagating through the large transformer encoders multiple times. Gloeckle et al. (2024) adopts a similar strategy to efficiently compute gradients of an encoder with multiple heads.

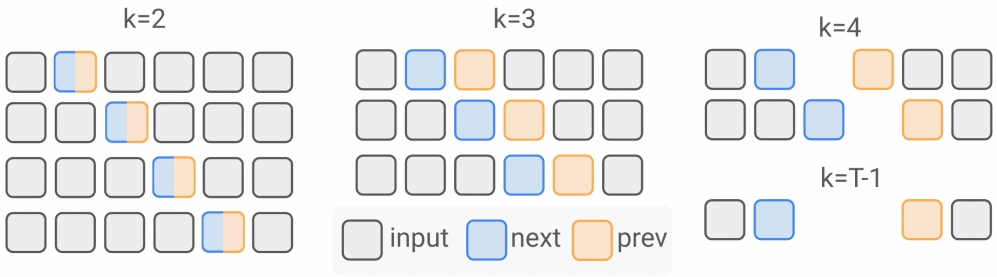


Figure 11: Forming training examples for the BST.

We define the training overheads. Let the sequence length be n. To scale with batch size, one would multiply the batch size with the following equations. Let h be the cost related to the output head size, and e be the cost of soft attention.

GPT: $O(hT + eT^2)$, first term is cost of doing $\texttt{text_head}(f_i)$ for $f_{1:T}$, second term is cost of encoding the tokens $x_{1:T}$ with attention to produce $f_{1:T}$.

BST: $O(hT^2 + eT^2)$, first term is cost of doing $\texttt{text_head}(f_i, b_j)$ for all valid pairs in the $O(T^2)$ prefix-suffix pairs, second term is cost of encoding the prefix and suffix tokens with attention. Note that at inference, BST is $O(hT + eT^2)$.